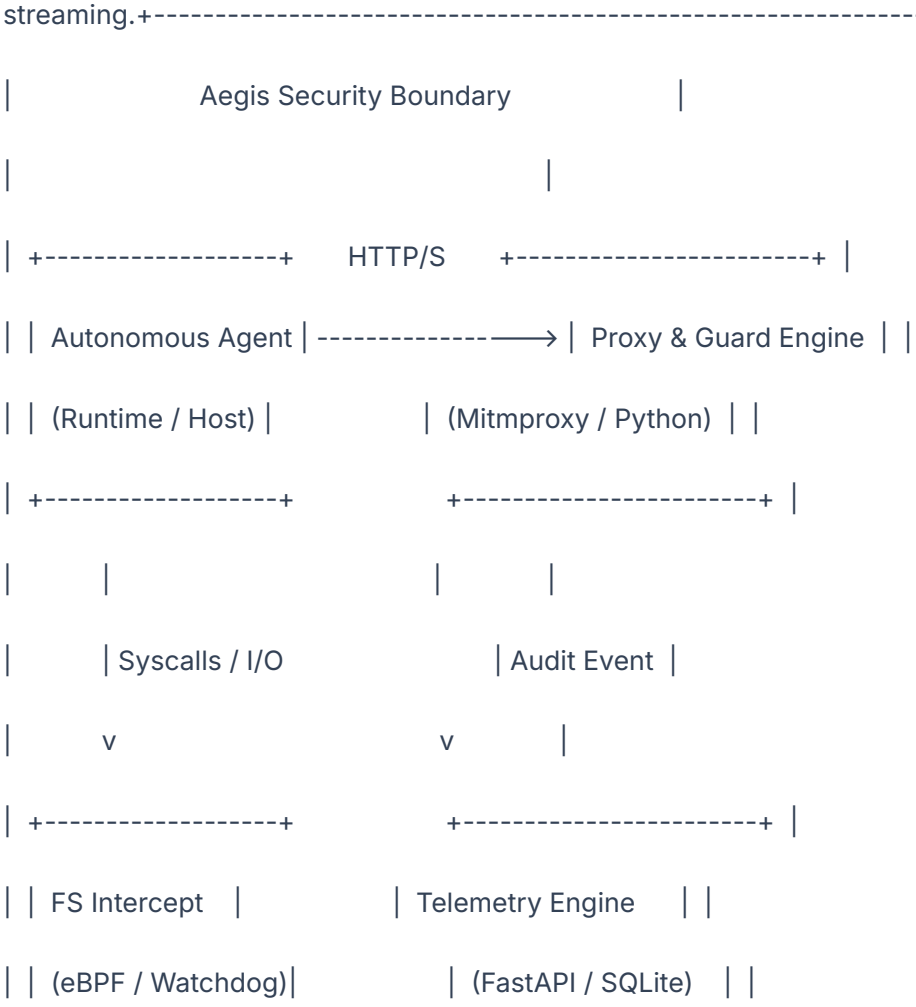
Aegis: Agentic Execution Guard & Inspection System

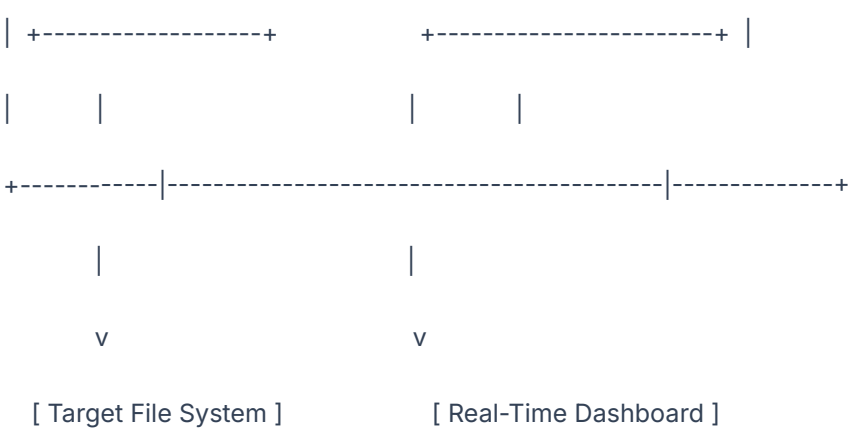
System Architecture Specification & Implementation Roadmap

Aegis is a lightweight, zero-trust security sidecar designed to monitor, filter, and inspect runtime operations initiated by autonomous AI agents. By intercepting outbound network requests, local file system operations, and execution environments, Aegis prevents rogue actions, data exfiltration, and unauthorized system access.

---

1. System Architecture

Aegis runs alongside target agent execution environments as an isolated sidecar container or host daemon. It enforces strict policies through intercepting proxies, file system hooks, and structured event
streaming. 



Key Architectural Components

- **In-Line Network Guard:** Intercepts HTTP, HTTPS, and WebSocket traffic. Evaluates domain allowlists, IP CIDR blocks, and payload structures prior to execution.
- **File System Supervisor:** Monitors local volume mounts and working directories using kernel-level triggers or file event API listeners to detect unauthorized file writes, executions, or directory traversals.
- **Security Policy Engine:** Central evaluation core processing rules written in declarative YAML formats. Matches runtime signatures against established constraints.
- **Telemetry & Dashboard Core:** Ingests intercepted log events, stores structured metrics, and exposes APIs for real-time visualization and administrative override.

---

2. Technology Stack

Layer	Component / Tool	Primary Function
Language Engine	Python 3.11+	Core guard logic, dynamic policy evaluation, API services
Network Proxy	`mitmproxy` / `httpx`	In-flight request interception, header modification, domain filtering
Containerization	Docker / Docker Compose	Isolation of sidecar components, predictable dev/prod parity
FS Monitoring	`watchdog` / eBPF (`bcc`)	Local file system event detection (creation, write, delete, execute)
Data Layer	SQLite / Redis	Low-latency audit logging, policy caching, event queueing

Layer	Component / Tool	Primary Function
Backend API	FastAPI	Async ingestion pipeline, control plane endpoints
Frontend UI	React / Tailwind CSS	Real-time monitoring dashboard, log viewer, policy editor

---

3. Implementation Roadmap

Phase 1: Core Proxy & Network Blocking

Focus: Intercepting and controlling egress agent communications.

- **Mitmproxy Integration:** Deploy a configured forward proxy intercepting outbound HTTP/HTTPS requests from the execution context.
- **Allowlist / Blocklist Engine:** Implement matching algorithms based on domains, full URIs, exact IP addresses, and HTTP verbs.
- **Dynamic Request Dropping:** Intercept malformed or non-compliant outbound requests, returning structured `403 Forbidden` responses to the calling agent.
- **TLS Unpacking (Development Mode):** Provide local CA certificate installation tooling to allow deep packet inspection of payload bodies.

Phase 1.3: Audit Logging System

Focus: Creating an audit trail of AI agent network behavior.

- **Structured JSON Logging:** Implement a structured JSON logging system to record both allowed and blocked network attempts, establishing a clear audit trail of AI agent behavior for downstream analysis in Phase 3.

Phase 2: File System Interception

Focus: Safeguarding local disk assets and workspace state.

- **Event-Driven File Monitoring:** Integrate OS-level hooks to observe changes within designated runtime directories.
- **Policy Rule Enforcer:** Restrict file access based on file extensions, absolute path patterns, write volumes, and file size limits.
- **Quarantine Mechanism:** Automatically isolate newly created executable binaries or unauthorized files prior to execution.
- **Integrity Hash Ledger:** Track hash changes across sensitive configuration files to detect unauthorized runtime tampering.

Phase 3: Data Analysis & Dashboard

Focus: Observability, alerting, and security metrics.

- **FastAPI Event Pipeline:** Stream security events, rule violations, and intercepted payloads to a central persistence layer.
- **Real-Time Monitoring Dashboard:** Develop a user interface displaying active agent sessions, block counts, network request streams, and alert status.
- **Threat Analytics & Heuristics:** Implement pattern detection for suspicious behavior loops (e.g., rapid egress scanning, directory crawling).
- **Policy Management Console:** Provide an administrative interface to modify rules and push updated configurations to active Aegis sidecars without downtime.

---

4. Policy Configuration Format

Aegis uses declarative YAML configurations to enforce rules across network and file systems.version: "1.0"

agent_id: "agent-researcher-01"

network_policy:

  default_action: "deny"

  allowed_domains:

    - "*.api.openai.com"

    - "github.com"

    - "api.github.com"

  blocked_ips:

    - "10.0.0.0/8"

    - "172.16.0.0/12"

  max_request_rate: 60 # per minute

filesystem_policy:

  watch_paths:

```
  - "/app/workspace"
read_only_paths:
  - "/app/config"
  - "/etc"
blocked_extensions:
  - ".sh"
  - ".exe"
  - ".bat"
max_file_size_mb: 50
```

---

5. Deployment Setup

Aegis is packaged as a lightweight Docker multi-container environment, isolating the guard system from the target runtime.# Clone repository

```
git clone https://github.com/example/aegis-guard.git
cd aegis-guard
# Initialize environment & certs
./scripts/setup_ca.sh
# Spin up sidecar core and dashboard
docker-compose up --build -d
# Execute agent through Aegis proxy
export HTTP_PROXY="http://localhost:8080"
export HTTPS_PROXY="http://localhost:8080"
python run_agent.py
```

6. Conclusion

The MVP of Aegis is now complete, marked by the successful implementation of the Streamlit dashboard providing real-time telemetry of network and filesystem events. This forms a solid foundation for further enhancements like Docker containerization.

Guida di Studio Dettagliata - Architettura e Codice Aegis

Questa sezione fornisce una guida di studio tecnica e approfondita sul sistema Aegis, analizzando l'architettura di sicurezza, la struttura del codice sorgente, i meccanismi di intercettazione a basso livello e le componenti di analisi dati.

1. Cos'è Aegis e qual è il problema reale che risolve

Aegis è una soluzione di sicurezza zero-trust progettata come sidecar di protezione per agenti IA autonomi. Gli agenti basati su Large Language Models (LLM) presentano vulnerabilità uniche dovute alla loro natura dinamica e alla capacità di eseguire codice e azioni sul sistema ospitante.

- **Prompt Injection (Diretta e Indiretta):** Un attaccante può manipolare gli input di testo o i dati letti dal modello (es. pagine web, file di testo) per deviare il comportamento dell'agente, inducendolo a eseguire comandi malevoli.
- **Data Exfiltration:** Previene il trasferimento non autorizzato di dati sensibili (chiavi API, credenziali, dati personali) verso server esterni sotto il controllo dell'attaccante.
- **Remote Code Execution (RCE):** Mitiga l'esecuzione involontaria di script, binari o comandi di sistema non verificati.
- **Zero-Trust Execution Mode:** Nessuna chiamata di rete o operazione I/O su file system è fidata a priori. Ogni richiesta viene intercettata, verificata rispetto alle policy e registrata in tempo reale.

2. Struttura del Codice e Pacchetti

Il progetto è organizzato nei seguenti moduli principali:

- `core/logger.py`: Gestisce la registrazione strutturata degli eventi di audit in formato JSONL (JSON Lines). Implementa un meccanismo sicuro per evitare loop infiniti durante la scrittura dei log.
- `core/network_guard.py`: Implementa l'intercettazione di rete a livello di socket tramite Monkey Patching, valutando le chiamate outbound rispetto a regole di allowlist e blocklist.
- `core/fs_guard.py`: Esegue il sovrascrittura (overriding) della funzione built-in `open` di Python per applicare restrizioni di lettura/scrittura e prevenire attacchi di Path Traversal.
- `test_agent.py`: Agente di test simulato per verificare il blocco attivo delle minacce sia a livello di rete che di file system.
- `app.py`: Dashboard interattiva sviluppata in Streamlit per l'analisi di telemetria, il monitoraggio delle violazioni e la visualizzazione analitica dei log.

3. Come funziona ogni riga/funzione chiave

Monkey Patching di socket.create_connection: Nel modulo `network_guard.py`, la funzione originale viene salvata (`_original_create_connection = socket.create_connection`) e sostituita con un wrapper di guardia. Quando il runtime dell'agente tenta di connettersi a un host e una porta, il wrapper risolve l'indirizzo IP, controlla la lista di domini e CIDR consentiti/bloccati e solleva un'eccezione `PermissionError` in caso di violazione prima di stabilire la connessione TCP.
Overriding di builtins.open: Nel modulo `fs_guard.py`, la funzione standard di apertura file viene intercettata. Prima di concedere il descrittore di file, vengono verificati la modalità di accesso (es. lettura, scrittura, append) e il percorso target.
Path Traversal Protection con os.path.commonpath: Per prevenire attacchi tramite percorsi relativi (es. `../../etc/passwd`), il percorso richiesto viene risolto in forma assoluta tramite `os.path.abspath`. Successivamente, si verifica che `os.path.commonpath([base_directory, target_path]) == base_directory`. Se il percorso comune non coincide con la cartella consentita, la richiesta viene bloccata.
Logging JSONL ed evitamento della ricorsione infinita: Durante la registrazione di un evento su file in `logger.py`, l'uso della funzione intercettata `open` causerebbe una ricorsione infinita (il logger chiama `open`, che chiama la guardia, che chiama il logger). Per evitare questo deadlock, il modulo di logging mantiene un riferimento diretto alla funzione built-in originale (`_original_open`) per la scrittura dei file JSONL di audit.

4. Dashboard di Telemetria in Streamlit e Pandas

La dashboard `app.py` offre una vista analitica avanzata per i team di Security Operation Center (SOC) e Data Analyst:

- **Integrazione Pandas DataFrame:** I log JSONL vengono caricati dinamicamente in un DataFrame Pandas, consentendo operazioni di filtraggio, aggregazione e pulizia in tempo reale.
- **Metriche e KPI:** Visualizzazione istantanea del numero totale di richieste intercettate, percentuale di eventi bloccati vs. consentiti e ripartizione per tipologia di risorsa (Network vs File System).
- **Analisi Visuale:** Grafici temporali sulle distribuzioni degli attacchi, heatmap delle destinazioni di rete bloccate ed esploratore di payload per l'ispezione forense degli eventi di minaccia.

5. Perché questo progetto valorizza le competenze da ingegnere e data analyst

Aegis dimostra una padronanza completa dello stack tecnologico moderno:

- **Ingegneria dei Sistemi e Sicurezza:** Capacità di operare a basso livello sul runtime Python, comprendendo i concetti di intercettazione di rete, gestione del file system e architetture sidecar isolati.
- **Data Engineering & Analytics:** Progettazione di pipeline di log strutturati e utilizzo di strumenti di analisi dati (Pandas, Streamlit) per trasformare log grezzi di sicurezza in intelligence operativa e metriche chiare.

- **Visione Produttiva e AI Governance:** Risposta concreta alle esigenze aziendali di deployment sicuro di agenti IA in ambienti enterprise trasparenti e conformi alle policy aziendali.